Comprehensive Testing Guide for the Library Management System

SE507 Group Project - Spring 2025

June 10, 2025

Abstract

This document provides a complete guide for testing the Library Management System, based on the Software Requirements Specification (SRS) and the established project tasks. It is designed to be used by both technical and non-technical team members, providing clear instructions and best practices for each phase of the testing lifecycle. The guide leverages modern technologies and methodologies discussed in the course lectures to ensure a high-quality, robust, and reliable final product.

1 Introduction

The Library Management System is a comprehensive software solution designed to automate and streamline the operations of a library. The successful implementation of this system relies on a rigorous and well-structured testing process. This guide is based on the provided project files, including the SRS, task distribution documents, and course lectures. It aims to provide a unified framework for all testing activities.

1.1 Purpose and Scope

The primary purpose of this guide is to detail the methodologies, tools, and best practices for testing the Library Management System. The scope encompasses all phases of testing, from initial static analysis to final user acceptance testing, ensuring that all functional and non-functional requirements outlined in the SRS are met.

1.2 Team Composition and Responsibilities

The testing team is composed of members with diverse skill sets, categorized as follows:

- **Coders:** Team members with strong programming and technical skills. They are primarily responsible for white-box testing, automation, and performance analysis.
- **Non-Coders/Specification Experts:** Team members who excel at user-centric evaluation and requirements analysis. They are responsible for black-box testing, usability testing, and ensuring the system aligns with user expectations.

A detailed breakdown of task distribution is provided in the "Task Distribution for Library Management Project Testing.docx" document and is summarized in the relevant sections of this guide.

2 Testing Strategy

The testing strategy is designed to be comprehensive, covering all aspects of the system. It integrates various testing levels and types to ensure maximum defect detection and quality assurance.

2.1 Testing Levels

The project will follow a standard V-Model approach to testing, with the following levels:

- **Unit Testing:** (Primarily performed by developers) To test individual components in isolation.
- **Integration Testing:** To test the interfaces and interactions between integrated components.
- **System Testing:** To test the complete, integrated system against the SRS.
- **Acceptance Testing:** To validate the system from a user's perspective.

2.2 Testing Types

Both functional and non-functional testing will be performed:

- **Functional Testing:** Validates the software against the functional requirements. This includes:
  - Use Case Testing
  - State Transition Testing
  - Entity Management Testing
  - Authentication and Authorization Testing
- **API Testing:** Focuses on testing the Application Programming Interfaces (APIs) directly.
- **Non-Functional Testing:** Validates aspects of the system not related to specific functions. This includes:
  - Performance Testing
  - Security Testing
  - Usability Testing
  - Browser Compatibility Testing
  - Maintainability Testing
- **Regression Testing:** To ensure that new changes do not negatively impact existing functionality.
- **Static Testing:** Involves reviewing code and documentation without executing the software.

3 Technological Stack and Tools

The project utilizes a modern technology stack that requires specific tools for effective testing.

3.1 Application Stack

- **Frontend:** React.js
- **Backend:** Node.js with Express.js
- **Database:** MongoDB
- **Containerization:** Docker

3.2 Testing Tools

- **API Testing:** Postman will be used for manual and automated API testing.
- **End-to-End Testing:** Cypress is the designated tool for automated end-to-end testing.
- **Performance Testing:** Tools like JMeter or k6 can be used to simulate load and measure performance.
- **Code Analysis:** Linters and static analysis tools integrated into the development environment.

3.3 Environment Setup

Before any testing can begin, the application environment must be running.

Coders: Your primary responsibility is to manage the Docker environment.

1. **Navigate to Root Directory:** Open a terminal in the project's root folder.
2. **Build and Run Containers:** Execute the command:

   ```
   docker-compose -f docker-compose.dev.yml up --build
   ```

   This command starts the frontend, backend, and database services as defined in the development compose file.
3. **Verify Services:**
   - The React frontend should be accessible at `http://localhost:3000`.
   - The Node.js backend API will be running on `http://localhost:5000`.
4. **Troubleshooting:** If containers fail to start, check the Docker logs for errors. Common issues include port conflicts or configuration errors in the '.yml' files.

Non-Coders: Your main setup task is to ensure you can access the application and have the necessary tools ready.

1. **Access the Application:** Once the Coders have confirmed the environment is up, open a web browser and navigate to `http://localhost:3000`.
2. **Install Postman:** If you don't have it, download and install Postman. This will be your primary tool for API testing.

4 Static Testing (Non-Execution Based)

Static testing is performed early in the lifecycle to find defects without executing code. It involves reviewing documentation and source code.

4.1 Documentation Review

Non-Coders/Specification Experts: You will lead the review of all non-technical and requirements-related documents.

- **Source Documents:** "SRS Library Management.docx", "Tasks.pdf", all "TC_*.tex" files.
- **Objective:** Check for clarity, consistency, completeness, and correctness. Ensure the test cases accurately reflect the requirements in the SRS.
- **Process:** Read through the documents, noting any ambiguities, contradictions, or missing information. Use a collaborative document or a simple spreadsheet to log your findings.

Coders: You will support the documentation review by focusing on technical accuracy.

- **Objective:** Review the SRS and test plans to ensure they are technically feasible and that the descriptions of system behavior are accurate from an implementation perspective.
- **Process:** Participate in review meetings led by the Non-Coders and provide feedback on technical sections.

4.2 Code Review

Coders: You will lead the technical review of the source code.

- **Source Files:** All '.js', '.jsx', and '.ts' files in the 'client/', 'server/', and 'e2e/' directories.
- **Objective:** Identify potential bugs, security vulnerabilities, and deviations from coding standards. Assess logic, efficiency, and maintainability.
- **Process:** Use a systematic approach. For example, for each file, check:
  1. Is the logic correct and efficient?
  2. Is there proper error handling?
  3. Are there any obvious security flaws (e.g., hardcoded secrets, injection vulnerabilities)?
  4. Does the code adhere to project-wide standards for naming and style?

Non-Coders/Specification Experts: Your role is to ensure the code is understandable and aligns with the specified requirements.

- **Objective:** Focus on readability, comments, and the high-level structure of the code. You are not expected to understand complex algorithms, but rather to assess if the code's purpose is clear and seems to match the user stories.
- **Process:** During review sessions (or "walkthroughs") led by the Coders, ask questions to clarify how the code achieves a specific feature described in the SRS. For example: "Can you show me the code that prevents a non-librarian from adding a book?"

5 Functional and API Testing

This phase focuses on verifying that the system behaves according to its specifications.

5.1 API Testing with Postman

Non-Coders/Specification Experts: Your primary role is to design the test scenarios based on the SRS.

1. **Document Reference:** `TC_API_Testing.tex`.
2. **Task:** For each API endpoint (e.g., 'POST /api/auth/login', 'GET /api/book/getAll'), define test cases for:
   - **Positive scenarios:** Testing with valid data (e.g., logging in with correct credentials).
   - **Negative scenarios:** Testing with invalid data (e.g., logging in with a wrong password, trying to fetch a book with an invalid ID).
   - **Edge cases:** Testing boundary conditions (e.g., empty request bodies, very long strings).
3. **Execution:** Using Postman, you will manually execute these scenarios. For a 'POST' request, you will set the URL, select the method, and provide the JSON body. After sending the request, verify that the HTTP status code (e.g., 200, 401, 404) and the response body match your expected results.

Coders: Your role is to automate the scenarios designed by the Non-Coders and to test more complex API interactions.

1. **Task:** Use Postman's test scripting features (or a framework like 'supertest' as seen in 'auth.api.test.js') to create automated test suites.
2. **Example Postman Test Script:** For the login endpoint, you can write a test to verify the response:

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
pm.test("Response contains an authentication token", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.token).to.not.be.empty;
});
```

3. **Chained Requests:** Create tests that chain requests together. For example, a test could first log in to get a token, then use that token in the header of a subsequent request to access a protected endpoint.

5.2 End-to-End (E2E) Testing with Cypress

E2E testing simulates real user scenarios from start to finish.

Coders: You are responsible for writing and maintaining the Cypress E2E tests.

1. **Source Files:** The primary files are in the ‘e2e/cypress/e2e/‘ directory, such as ‘authentication_authorization.cy.ts‘.
2. **Running Tests:**
   - **Interactive Mode:** Use this for writing and debugging tests.

     ```
     # From the root directory
     npm run e2e:watch
     ```

   - **Headless Mode:** Use this for running the full test suite (e.g., in a CI/CD pipeline).

     ```
     # From the root directory
     npm run e2e
     ```

3. **Writing Tests:** Follow the existing patterns in ‘authentication_authorization.cy.ts‘. Use descriptive ‘describe‘ and ‘it‘ blocks. Use commands like ‘cy.visit()‘, ‘cy.get()‘, ‘cy.click()‘, and ‘cy.type()‘ to interact with the UI. Use assertions like ‘.should(’exist’)‘ or ‘.should(’contain’, ’text’)‘ to verify outcomes.

Non-Coders/Specification Experts: You are responsible for designing the E2E test scenarios and validating the results.

1. **Task:** Based on the Use Cases in the SRS (e.g., ‘UC-002: Add New Book‘), write out the step-by-step user actions and expected outcomes in plain language.
2. **Example Scenario (Add New Book):**
   (a) Login as a Librarian.
   (b) Navigate to the ”Book Management” page.
   (c) Click the ”Add Book” button.
   (d) Fill in the book title, author, genre, and other details in the form.
   (e) Click ”Save”.
   (f) **Expected Result:** The new book appears in the list on the ”Book Management” page. A success notification is shown.
3. **Collaboration:** Provide these scenarios to the Coders to be automated. You will then review the execution of the automated tests (either by watching them run or by checking the reports) to confirm they match the intended user flow.

6 Non-Functional Testing

This phase tests system attributes like performance, security, and usability.

6.1 Usability Testing

Non-Coders/Specification Experts: You will lead usability testing, as it is entirely user-focused.

- **Document Reference:** 'TC_Usability_Testing.tex'.
- **Objective:** Evaluate how easy and intuitive the system is to use.
- **Process (Heuristic Evaluation):** Go through the application and assess it against established usability principles:
  - **Consistency:** Are buttons, menus, and terminology consistent across all pages?
  - **Feedback:** Does the system provide clear feedback for actions (e.g., showing a "loading" spinner, a "success" message)?
  - **Clarity:** Is the text clear and unambiguous? Are icons easy to understand?
  - **Error Prevention:** Is it easy for users to make mistakes? How does the system help prevent them?
- **Reporting:** Document findings with screenshots and detailed descriptions of any confusing or difficult-to-use parts of the interface.

6.2 Security Testing

Coders: You are responsible for technical security testing.

- **Document Reference:** 'TC_Security_Testing.tex'.
- **Key Areas to Test:**
  - **Authorization Bypass:** Use Postman to try accessing a librarian-only API endpoint (e.g., 'POST /api/book/add') using the authentication token of a regular member. The expected result is a '403 Forbidden' status code.
  - **Data Exposure:** Check the responses of all API endpoints to ensure they are not leaking sensitive information (e.g., the user list endpoint should not return password hashes).

6.3 Performance Testing

Coders: You will lead performance testing using appropriate tools.

- **Document Reference:** 'TC_Performance_Testing.tex'.
- **Tool:** Apache JMeter or k6.
- **Objective:** Determine how the system behaves under load.
- **Process (Load Test):**
  1. Create a test plan in JMeter.
  2. Define a Thread Group to simulate a number of concurrent users (e.g., 50 users).
  3. Add HTTP Request samplers for key API endpoints (e.g., 'GET /api/book/getAll').
  4. Add listeners to view the results (e.g., Summary Report, Aggregate Graph).
  5. Run the test and analyze the results, focusing on average response time and error rate. The system should maintain acceptable response times without a high error percentage.

7 Regression Testing

Regression testing is crucial to ensure that new features or bug fixes have not broken existing functionality.

Coders: You are responsible for running the automated regression suite.

- **Process:** After any significant code change, run the entire Cypress E2E test suite and the automated Postman/API test suite.
- **Objective:** All tests should pass. Any failures indicate a regression that must be fixed before the changes are accepted.

Non-Coders/Specification Experts: You are responsible for performing manual, risk-based regression testing.

- **Document Reference:** 'TC_Regression_Testing.tex'.
- **Process:** Based on the recent changes, identify the areas of the application most at risk. For example, if a change was made to the borrowing logic, you should manually re-test:
  - Borrowing a book.
  - Trying to borrow an already borrowed book.
  - Viewing borrowal history.
  - Returning a book.
- **Objective:** To provide an additional layer of confidence by testing scenarios that may not be covered by automation.

8 Advanced Functional Testing Scenarios

This section expands on the core functional testing by introducing model-based techniques and a deeper focus on data integrity, as discussed in the lectures.

8.1 Entity Management and Data Integrity Testing

This testing ensures that the relationships between data entities (Books, Authors, etc.) are correctly enforced by the system.

Non-Coders/Specification Experts: You are responsible for designing the test cases based on the data model in the SRS.

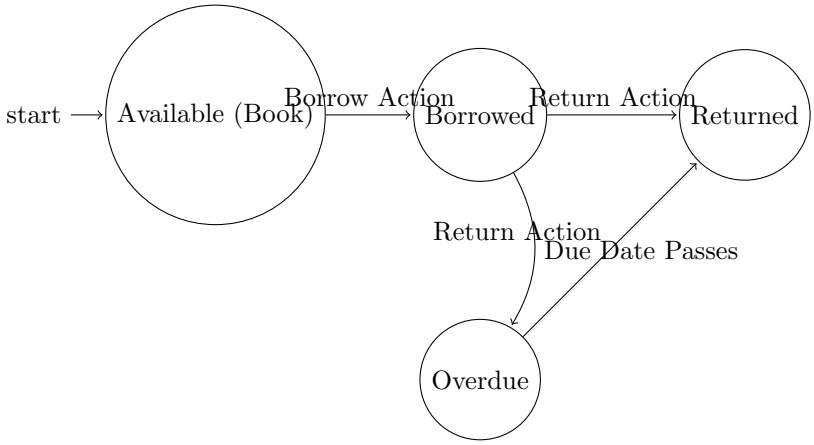
- **Document Reference:** 'TC_Entity_Management.tex'.
- **Objective:** To verify that all Create, Read, Update, and Delete (CRUD) operations work as expected and that data constraints are enforced.
- **Task:** For each entity, design tests for scenarios like:
  - Attempting to create a 'Book' without selecting a valid 'Author' or 'Genre'. The system should show an error.
  - Attempting to delete an 'Author' who still has 'Book's associated with them in the system. This should be prevented.
  - Updating a 'User''s role and verifying their permissions change accordingly.
- **Execution:** These tests are typically run manually through the UI, checking that the system behaves as you've designed.

8.2 State Transition Testing

As covered in the lectures, systems can often be modeled as a Finite State Machine (FSM). This technique tests the transitions between different states of an entity. The 'Borrowal' entity is a perfect candidate.

Non-Coders/Specification Experts: Your role is to model the entity's lifecycle and design tests to cover it.

- **Document Reference:** 'TC_State_Transition_Testing.tex'.
- **Modeling:** First, define the states and transitions for a borrowal.



- **Test Design:** Create test cases that trigger each transition. For example:
  - **Test 1 (A to B):** Log in as a member, find an available book, and execute the borrow function. Verify the book's status changes to "Borrowed".
  - **Test 2 (B to D):** For a borrowed book, manually change the system date past the due date (a Coder may need to assist with this) and verify the status becomes "Overdue".
  - **Test 3 (Invalid Transition):** Attempt to borrow an already "Borrowed" book. The system should display an error.

Coders: Your task is to automate these state transition tests where feasible.

- **Objective:** To create robust tests that can programmatically drive an entity through its lifecycle.
- **Process:** This often requires a combination of API calls. For the borrowal example, an automated test script could:
  1. Call the login API to get a member token.
  2. Call the 'borrow' API endpoint.
  3. Call a special debug endpoint (that you might create) to advance the time on the server.
  4. Call the 'getBorrowal' API to check that the status is now 'Overdue'.
  5. Call the 'return' API.
  6. Call 'getBorrowal' again to check that the status is 'Returned'.

9 Integration Testing

Integration testing focuses on verifying the interaction between different components or modules of the system.

Coders: You will lead the implementation of integration tests, as they are primarily code-based.

- **Document Reference:** 'TC_Integration_Testing.tex' and the existing test 'server/__tests__/integration/auth.api.tes
- **Objective:** Test the data flow and communication between the Node.js backend API and the MongoDB database.

- **Process:** Write tests using a framework like 'supertest' and 'jest'. These tests make real API calls to the server and can check the database state to verify the outcome.
- **Example Integration Test Scenario:**
  1. The test starts by clearing the relevant database collections.
  2. It makes a 'POST /api/author/add' request to create a new author.
  3. It then directly queries the database to assert that the author document was created correctly.
  4. It makes a 'POST /api/book/add' request, using the new author's ID.
  5. It queries the database to assert that the book was created and correctly references the author.

Non-Coders/Specification Experts: You will support integration testing by defining the cross-module user workflows.

- **Task:** Identify user journeys that touch multiple parts of the system.
- **Example Scenario:** "A member searches for a book, finds it, adds a review for it, and then borrows it. This workflow integrates the Search, Review, and Borrowal modules."
- **Collaboration:** Provide these high-level scenarios to the Coders to guide the creation of their technical integration tests.

10 Finalizing the Testing Process

10.1 Browser Compatibility Testing

Non-Coders/Specification Experts: You will lead the manual execution of these tests.

- **Document Reference:** 'TC_Browser_Compatibility_Testing.tex'.
- **Objective:** To ensure the application looks and works correctly on major web browsers.
- **Process:**
  1. Identify the target browsers (e.g., latest versions of Chrome, Firefox, and Safari/Edge).
  2. Execute a set of key test cases (a "smoke test" suite) on each browser. This should include core functionality like login, search, and viewing data tables.
  3. Look for any visual bugs (e.g., misaligned elements, broken layouts) or functional issues (e.g., buttons not working).
  4. Document any differences in behavior with screenshots.

10.2 Defect Management and Reporting

A structured approach to managing defects is essential for project success.
Both Coders and Non-Coders: Defect reporting is a shared responsibility.

1. **Finding a Defect:** When a test case fails (either manual or automated), a defect has been found.
2. **Logging the Defect:** Create a defect report using a simple tool (like a shared spreadsheet, Trello, or Jira). The report MUST contain:
   - A clear, concise title (e.g., "Login fails with valid member credentials").
   - The environment it was found in (e.g., Local dev, Chrome).
   - Detailed, numbered steps to reproduce the bug.
   - The **Expected Result** (what should have happened).
   - The **Actual Result** (what did happen).
   - Screenshots or log files.
   - An initial assessment of Severity (how much impact it has on the system) and Priority (how urgently it should be fixed).

Coders: You are responsible for resolving defects.

1. Pick up a defect from the log.
2. Analyze the issue and implement a code fix.
3. Update the defect report, marking it as "Ready for Retest".

Non-Coders/Specification Experts: You are responsible for verifying fixes.

1. Take a defect marked "Ready for Retest".
2. Execute the original test case that found the bug.
3. If the bug is gone, close the defect report. If it still exists, re-open it and assign it back to the Coders with a comment.

11 Maintainability Testing

Maintainability testing assesses the ease with which the system can be understood, modified, and enhanced. This is a crucial non-functional requirement for the long-term health of the project.

Coders: You are responsible for the technical analysis of the codebase's maintainability.

- **Document Reference:** 'TC_Maintainability.tex'.
- **Objective:** To measure the structural quality and complexity of the code.
- **Key Metrics and Tools:**
  - **Code Coverage:** Use the built-in Jest test coverage reporting. After running tests, check the coverage report. A higher percentage indicates that more code is exercised by tests, making it safer to modify.

```
# From the /server directory
npm test -- --coverage
```

  - **Code Complexity:** Manually assess the cyclomatic complexity of critical functions (e.g., in 'borrowalController.js'). A function with many nested 'if'/'else' statements or loops has high complexity and is harder to maintain. Aim to refactor highly complex functions into smaller, simpler ones.
  - **Static Analysis:** Pay close attention to warnings from ESLint. These tools are designed to catch "code smells" that can indicate poor maintainability.

Non-Coders/Specification Experts: You will assess maintainability from a documentation and knowledge transfer perspective.

- **Objective:** To evaluate how easy it would be for a new team member to understand the system.
- **Process:**
  - **Documentation Quality:** Review the code comments, the 'README.md' files, and the SRS. Is the documentation clear, accurate, and up-to-date?
  - **Traceability:** Can you trace a requirement from the SRS to a specific test case and, with a Coder's help, to the part of the code that implements it? Strong traceability is a sign of a maintainable system.
  - **Configuration Management:** Review the 'docker-compose' files. Are the configurations easy to understand? Is it clear how to start the system in different environments (dev, test, prod)?

12 Test Metrics and Exit Criteria

To manage the testing process effectively and to know when to stop, the team must define and track key metrics and agree on "exit criteria".

12.1 Key Performance Indicators (KPIs) for Testing

Both Coders and Non-Coders: The entire team is responsible for tracking and discussing these metrics in regular meetings. A simple shared spreadsheet can be used as a dashboard.

- **Requirements Coverage:** What percentage of requirements defined in the SRS have at least one test case associated with them? The goal is 100%.
- **Test Case Pass Rate:** The percentage of executed test cases that have passed.

$$\text{Pass Rate} = \left(\frac{\text{Number of Passed Tests}}{\text{Total Number of Executed Tests}} \right) \times 100\%$$

- **Defect Density:** The number of confirmed defects per module or feature. This helps identify which parts of the application are most unstable.
- **Defect Discovery Rate (DDR):** The number of defects found per day or week. A falling DDR towards the end of the cycle can indicate that the system is stabilizing.

12.2 Test Exit Criteria

These are the conditions that must be met for the team to formally conclude the testing phase.

- A Test Case Pass Rate of at least **95%** has been achieved.
- Requirements Coverage is **100%**.
- All automated E2E and Integration test suites run to completion with **100%** of tests passing.
- There are **zero** open defects with a "Blocker" or "Critical" severity level.
- All "High" severity defects have a documented plan for being fixed, or a formal decision has been made to accept the risk.
- The Final Test Summary Report has been reviewed and approved by the entire team.

13 Final Test Summary Report

This is the final document produced by the testing team. It provides a comprehensive overview of all testing activities and gives a final assessment of the software's quality.

Non-Coders/Specification Experts: You will lead the compilation of this report, with Coders contributing to the technical sections.

13.1 Structure of the Report

The report should contain the following sections:

1. **Introduction:** A brief overview of the project and the scope of the testing conducted.
2. **Testing Scope and Approach:** A summary of the testing types performed (Functional, API, Performance, etc.) and the tools used.
3. **Test Results and Metrics:**
   - A summary of the final test metrics (Requirements Coverage, Pass Rate, etc.).
   - A summary of the results for each type of testing performed. For example, a section on Performance Testing should state the average response times under load.
4. **Defect Summary:**
   - An overview of the total number of defects found, categorized by severity (Blocker, Critical, High, Medium, Low).

- A list of any critical or high-severity defects that are still open, along with a justification if they are not being fixed.

5. **Overall Quality Assessment:** The team's final, professional judgment on the quality of the system. This section should explicitly state whether the system meets the quality goals defined at the start of the project.
6. **Recommendation:** A clear recommendation on whether the software is ready for "release" (i.e., project submission) based on the test results and exit criteria.
7. **Lessons Learned:** A brief section detailing what went well in the testing process and what could be improved in future projects.

A Appendix: Quick Reference

A.1 Key Commands

Run Dev Environment: `docker-compose -f docker-compose.test.yml up --build`

Run Cypress (Interactive): `npm run e2e:watch`

Run Cypress (Headless): `npm run e2e`

Run Backend Tests: `cd server && npm test`

A.2 Glossary of Terms

Black-Box Testing: Testing a system without knowledge of its internal workings.

White-Box Testing: Testing based on knowledge of the internal logic of an application's code.

Regression Testing: Re-running functional and non-functional tests to ensure that previously developed and tested software still performs after a change.

State Transition Testing: A black-box technique used to test the behavior of a system as it moves between different states.

SRS: Software Requirements Specification.